Assignment 1: Rescue Robot Planning

Due: 09/18/2026, 11:59 PM
Submission: On Canvas

Overview

In this assignment, you will formulate and solve a state-space search problem using both uninformed and informed search algorithms. You will implement and compare Breadth-First Search (BFS), Depth-First Search (DFS), Uniform-Cost Search (UCS), and A* Search. The goal is to understand how a problem can be represented as a state space and how different search strategies affect solution quality and computational performance.

Problem: Rescue Robot

A rescue robot must navigate through a building, retrieve a medical kit, and then reach an injured person. The environment is:

$$\begin{array}{ccccccc} S & . & . & \# & . & . & . \\ . & \# & . & \# & . & \# & . \\ . & \# & . & . & . & \# & G \\ . & . & \# & \# & . & . & . \\ . & K & . & . & . & \# & . \end{array}$$

Here, S denotes the starting location, K denotes the medical kit, G denotes the injured person, # denotes a blocked cell, and . denotes an open cell. The robot can move one cell at a time using one of four actions: *Up*, *Down*, *Left*, or *Right*. The robot may not move outside the grid or through a blocked cell. The robot must retrieve the medical kit before reaching the injured person.

Part 1: State-Space Representation

Represent each state as

$$s = (r, c, k),$$

where r is the robot's row, c is the robot's column, and $k \in \{\texttt{False}, \texttt{True}\}$ indicates whether the medical kit has been retrieved. For example, the state $(2, 3, \texttt{False})$ means that the robot is located at row 2, column 3 and has not yet retrieved the medical kit. The initial state is the location of S with $k = \texttt{False}$. When the robot enters the cell containing K, the value of k becomes `True`. The goal condition is satisfied when the robot reaches G and $k = \texttt{True}$.

Part 2: Successor Function

Implement the function `get_neighbors(state)`. The function should return all valid states that can be reached from the current state in one action. A move is valid if:

1. the destination is inside the grid, and
2. the destination is not a blocked cell.

If the robot enters the location containing the medical kit, update k to `True`. Use the following fixed action order whenever generating successors:

$$\text{Up} \rightarrow \text{Right} \rightarrow \text{Down} \rightarrow \text{Left}.$$

Using a fixed action order ensures that all implementations produce comparable results.

Part 3: Breadth-First Search

Implement `bfs(start_state)`. Breadth-First Search should use a FIFO queue to determine which state to expand next. Your implementation should keep track of previously visited states to avoid cycles. Your function should return the solution path and the number of states expanded.

```
return path, nodes_expanded
```

The path should contain the sequence of states from the initial state to the goal.

Part 4: Depth-First Search

Implement `dfs(start_state)`. Depth-First Search should use a stack or recursion. Your implementation must keep track of previously visited states to avoid cycles. Use the same fixed successor order:

$$\text{Up} \rightarrow \text{Right} \rightarrow \text{Down} \rightarrow \text{Left}.$$

Your function should return

`return path, nodes_expanded`.

Part 5: Introducing Path Costs

For Uniform-Cost Search and A* Search, use the following weighted version of the environment:

S	1	1	#	1	1	1
1	#	1	#	4	#	1
1	#	1	1	4	#	G
1	1	#	#	4	1	1
1	K	1	1	1	#	1

Each number represents the cost of entering that cell. Entering a cell labeled 1 has cost 1, while entering a cell labeled 4 has cost 4. Entering S, K, or G has cost 1. If a path consists of actions with costs $c_1, c_2, \ldots, c_m$, then its total cost is

$$C = \sum_{i=1}^{m} c_i.$$

Part 6: Uniform-Cost Search

Implement `ucs(start_state)`. Uniform-Cost Search should expand the state with the lowest accumulated path cost. For a state n, let $g(n)$ denote the cost of the path from the initial state to n. Use a priority queue to manage the search frontier, with priority determined by $g(n)$. Your function should return

`return path, path_cost, nodes_expanded`.

Part 7: A* Search

Implement `astar(start_state)`. A* Search prioritizes states according to

$$f(n) = g(n) + h(n),$$

where $g(n)$ is the path cost from the initial state to n, and $h(n)$ estimates the remaining cost required to complete the rescue. Use Manhattan distance as the basis for the heuristic. For two grid locations $p_1 = (r_1, c_1)$ and $p_2 = (r_2, c_2)$, define

$$D(p_1, p_2) = |r_1 - r_2| + |c_1 - c_2|.$$

Let x denote the robot's current position. If the medical kit has not yet been collected, use

$$h(n) = D(x, K) + D(K, G).$$

If the medical kit has already been collected, use

$$h(n) = D(x, G).$$

Implement this heuristic in the function `heuristic(state)` and use it within your A* implementation. Your function should return

`return path, path_cost, nodes_expanded`.

Part 8: Performance Comparison

Run all four algorithms on the provided environment.

For each algorithm, record:

- solution length,
- total path cost,
- number of states expanded, and
- runtime.

Measure runtime using `time.perf_counter()`.

Complete the following table:

Algorithm	Solution Length	Path Cost	States Expanded	Runtime
BFS				
DFS				
UCS				
A*				

Use the same environment and the same successor ordering for all algorithms.

For BFS and DFS, the search procedure itself should ignore the terrain costs. However, after obtaining the solution path, compute its total path cost using the same movement costs used by UCS and A*. This allows all four algorithms to be compared on the same problem.

Part 9: Analysis Questions

Answer each question in approximately 2–4 sentences.

1. Why does the state representation include k, indicating whether the robot has retrieved the medical kit, instead of representing a state using only the robot's row and column?
2. Did BFS and DFS return the same path? Explain why or why not.
3. Did BFS necessarily find the solution with the lowest total path cost? Explain the difference between minimizing the number of actions and minimizing total path cost.
4. Compare UCS and A*. Did they return solutions with the same total cost? Explain your result.
5. Which algorithm expanded fewer states: UCS or A*? Explain how the heuristic affects the search.
6. Suppose all actions had the same cost. Which search algorithm would be appropriate for finding the shortest path? How does your answer change when actions have different costs?

Required Functions

Your notebook should contain at least the following functions:

- `get_neighbors(state)`
- `bfs(start_state)`
- `dfs(start_state)`
- `ucs(start_state)`
- `heuristic(state)`
- `astar(start_state)`

You may define additional helper functions as needed.

Submission Requirements

Submit a single Jupyter Notebook or Google Colab notebook containing:

1. the state-space representation,
2. successor function,
3. BFS implementation,
4. DFS implementation,
5. UCS implementation,
6. A* implementation and heuristic,
7. performance results,
8. completed comparison table, and
9. responses to all analysis questions.

The notebook should execute correctly from beginning to end.

Grading

Component	Points
State-space representation and successor function	15
Breadth-First Search	15
Depth-First Search	15
Uniform-Cost Search	15
A* Search and heuristic	20
Performance comparison	10
Analysis questions	10
Total	**100**